

Sorbonne Université
Master AI2D

Assistance à l'utilisation de logiciels complexes à l'aide de VLM

UE de projet M1

Çagla KOPRULU – Riadh OUAZIB – Andy NGO – Louiza CHENAOUI

2026

Table des matières

1 Introduction **1**

2 État de l'art **2**
2.1 Approches CNN pour la classification de commandes 2
2.2 Modèles Vision-Langage (VLM) et choix du modèle 3
2.3 CNN vs VLM : comparaison 3
2.4 IA centrée sur l'humain et apprentissage interactif 4

3 Contribution **5**
3.1 Construction et évolution du dataset 5
3.2 Stratégie de fine-tuning 10
3.3 Plugin GIMP et serveur d'inférence 12
3.4 Analyse des résultats (tests gimp vs metriques) 14

4 Conclusion **16**
4.1 Bilan du projet 16
4.2 Limitations 16
4.3 Perspectives futurs 17

Annexe A : Cahier des charges **19**

Annexe B : Manuel utilisateur **24**

Chapitre 1

Introduction

L'apprentissage de l'utilisation de logiciels complexes par l'utilisateur, tels que des IDE comme VSCode ou des logiciels de traitement d'image comme GIMP, a été et est toujours une partie difficile de l'expérience utilisateur. Une étape importante de cet apprentissage, et qui est souvent manquée par la plupart des utilisateurs lambda est la connaissance des commandes dont le logiciel dispose et de leurs usage.

On peut citer par exemple le logiciel AutoCAD, un logiciel de dessin assisté par ordinateur permettant de créer des dessins techniques qui entre sa date de sortie sur MacOS en 1992 et l'année 2011. En 2009, plus de 50% de ses utilisateurs ne connaissaient au plus que 50 commandes sur les presque 750. Cette méconnaissance du logiciel par l'utilisateur handicape à la fois sa capacité à accomplir de nouvelles tâches, mais aussi à effectuer des tâches existantes plus rapidement.

Une solution qui a été explorée combine une base de données construite à partir des autres utilisateurs constituée des fréquences d'utilisation des commandes par ceux-ci, combiné à un algorithme de recommandation qui se base sur les commandes que l'utilisateur a employé.[1]

Une autre approche est de considérer chaque action de l'utilisateur comme étant un état, et d'essayer de déterminer quelle suite de commandes permettrait d'arriver à l'état final. On peut ainsi espérer faire des suggestions qui sont en lien direct avec les actions de l'usager et qui lui sont utiles, quelque chose qui a été exploré et implémenté pour l'application de traitement d'images Photoshop avec un CNN pour effectuer la post-diction dans ce projet AI2D.[2] Le but de ce projet est de voir si employer un VLM à la place du CNN permet d'obtenir de meilleurs performances.

Nos encadrants de projets sont Julien GORI, Olivier SCHWANDER et Gilles BAILLY, le dépôt Git est situé à https ://gitlab.com/Ardrets1/pai2d-2026.

Chapitre 2

État de l'art

Notre projet se situe à l'intersection de trois domaines : la vision par ordinateur, les modèles de langage multimodaux, et la conception d'interfaces centrées sur l'humain. Cette section présente les travaux existants dans chacun de ces domaines et positionne nos choix techniques par rapport à l'état de l'art.

2.1 Approches CNN pour la classification de commandes

Les travaux fondateurs sur lesquels notre projet s'appuie directement introduisent formellement le concept de postdiction de commandes. L'idée centrale est de comparer l'état courant de l'interface avec ses états précédents pour inférer quelle commande avancée aurait pu produire le même résultat plus efficacement, plutôt que de tenter de prédire ce que l'utilisateur veut faire dans le futur.

Pour le cas d'étude de l'éditeur d'images sur Photoshop, les auteurs implémentent un classifieur entraîné composé de trois instances ResNet pré-entraînées sur ImageNet, combinées par apprentissage ensembliste [2]. Le classifieur prend en entrée une concaténation horizontale de l'image source et de l'image cible redimensionnées à 122×244 pixels, et classifie parmi 8 filtres globaux. Avec un seuil de confiance à 0 %, il atteint 74 % de précision et 96 % de rappel ; en relevant le seuil à 95 %, les faux positifs diminuent de 36 à 19, au prix d'une légère baisse des vrais positifs.

Cette approche présente des avantages : les modèles CNN sont légers, rapides à l'inférence, et leur comportement est interprétable via les matrices de confusion. Cependant, elle montre aussi des limites structurelles importantes. Le nombre de classes est fixé à la conception, ce qui rend le système difficile à étendre. Les images sont fortement sous échantillonnées, ce qui peut effacer des détails fins nécessaires à la classification. Surtout, le modèle n'a aucune compréhension sémantique de ce qu'il observe, il détecte des patterns visuels sans raisonner sur leur signification. C'est précisément pour dépasser ces limites que notre projet explore une approche basée sur les VLMs, capables de raisonner sémantiquement sur les transformations visuelles.

2.2 Modèles Vision-Langage (VLM) et choix du modèle

Les modèles Vision-Langage (VLMs) combinent un encodeur visuel et un grand modèle de langage (LLM). L'encodeur visuel, généralement un Vision Transformer (ViT), extrait des représentations denses de l'image sous forme de patches. Ces représentations sont ensuite projetées dans l'espace du LLM via un module de connexion, permettant au modèle de raisonner simultanément sur le texte et le contenu visuel. Contrairement aux CNN classiques, les VLMs ne se limitent pas à reconnaître des motifs visuels : ils peuvent décrire, comparer et interpréter des images en langage naturel, ce qui les rend particulièrement adaptés à une tâche de classification où la distinction entre commandes repose sur des différences visuelles subtiles.

Nous avions le choix entre plusieurs modèles open-source pour notre projet : LLaVA 1.5 (7B), PaliGemma (3B) et InternVL2 (2B). LLaVA 1.5 a été écarté en raison de son empreinte mémoire trop importante pour notre contrainte d'entraînement sur deux GPU T4 de Kaggle. PaliGemma, bien que plus léger, s'est montré moins performant sur les tâches de comparaison visuelle entre deux images. Notre choix s'est finalement porté sur Qwen2-VL-2B-Instruct.

Ce modèle possède une architecture native multimodale ainsi qu'un tokenizer visuel adaptatif traitant jusqu'à 512 patches par image. Il offre de très bonnes performances sur les tâches de comparaison visuelle, s'adapte à nos contraintes matérielles et prend en charge notre format d'entrée en triptyque.

2.3 CNN vs VLM : comparaison

Ces deux approches répondent au même problème, classifier une transformation visuelle à partir d'une paire d'images, mais avec des philosophies fondamentalement différentes.

Les CNN sont rapides, légers et faciles à déployer. Leur entraînement est peu coûteux et leur comportement relativement prévisible. En revanche, ils apprennent des corrélations statistiques entre pixels sans aucune compréhension de ce qu'ils observent. Étendre le système à de nouvelles commandes implique de réentraîner entièrement le modèle, et les performances se dégradent rapidement quand le nombre de classes augmente ou que les différences visuelles entre classes sont subtiles.

Les VLMs, à l'inverse, apportent une compréhension sémantique de l'image. Grâce à leur pré-entraînement massif sur des données texte-image, ils savent déjà ce qu'est un flou, une inversion de couleurs, ou un contour détecté, avant même d'être fine-tunés sur nos données. Cela leur permet de mieux généraliser, notamment sur des classes visuellement proches comme brightness et contrast, où un CNN se perd facilement. En revanche, ils sont beaucoup plus lourds, plus lents à l'inférence, et nécessitent des techniques de quantification comme QLoRA pour être entraînables sur du matériel grand public.

Donc le choix d'un VLM implique un coût computationnel plus élevé, mais offre une capacité de raisonnement et de généralisation que les CNN ne peuvent pas atteindre dans le cadre de notre tâche.

2.4 IA centrée sur l'humain et apprentissage interactif

L'apprentissage automatique interactif (*Interactive Machine Learning*, IML) désigne les systèmes où l'humain et le modèle collaborent en boucle : l'utilisateur fournit des retours, le modèle s'adapte en temps réel. Ce paradigme, formalisé notamment par Amershi et al.[3] et Fiebrink[4], place l'utilisateur au centre du processus d'apprentissage plutôt qu'en simple destinataire d'un système figé.

Notre projet ne relève pas de l'IML au sens strict, le modèle ne se réentraîne pas en temps réel à partir des retours utilisateur. En revanche, nous nous inspirons de ses principes dans la conception de notre interface. En particulier, les travaux de Fiebrink[4] soulignent qu'une mauvaise suggestion nuit davantage à la confiance de l'utilisateur que l'absence de suggestion. Cela justifie notre choix d'une approche *precision-first* : nous préférons ne rien suggérer plutôt que de proposer une commande incorrecte, même si cela réduit mécaniquement notre taux de rappel.

Cela soulève également une question plus large sur l'évaluation de ce type de système. Les métriques classiques comme l'accuracy ou le F1-score mesurent la performance du modèle sur un dataset de validation, mais ne reflètent pas nécessairement l'utilité perçue par l'utilisateur. Une suggestion correcte au bon moment a plus de valeur qu'un score agrégé élevé sur des cas que l'utilisateur ne rencontrera jamais. Notre panneau GTK constitue un premier pas vers une évaluation plus centrée sur l'usage réel, dans la lignée de ce que Dudley et Kristensson[5] appellent les métriques d'évaluation non conventionnelles pour les systèmes d'assistance.

Chapitre 3

Contribution

3.1 Construction et évolution du dataset

Génération automatique de données synthétiques

La reconnaissance automatique d'une opération d'édition d'image à partir de son effet visuel constitue une tâche pour laquelle il n'existe pas de corpus public. Aucun jeu de données ne fournit de paires *avant/après* annotées pour les commandes d'un logiciel de retouche tel que GIMP. La première décision méthodologique a donc consisté à constituer un jeu de données entièrement synthétique, en appliquant automatiquement des commandes GIMP à un ensemble d'images sources via l'API Python-Fu et en enregistrant le résultat de chaque transformation.

Cette approche garantit des étiquettes exactes par construction, sans annotation manuelle ni ambiguïté. Elle introduit en revanche une contrainte : la distribution des données d'entraînement est entièrement déterminée par les choix de génération (images sources, paramètres, format composite), ce qui crée un risque de décalage avec les conditions réelles d'utilisation du plugin.

Le pipeline de génération est illustré en figure 3.1. Des scripts d'automatisation pilotent GIMP, appliquent une commande à chaque image source, et sauvegardent la paire (état avant, état après) accompagnée de l'étiquette textuelle de la commande et de ses paramètres.

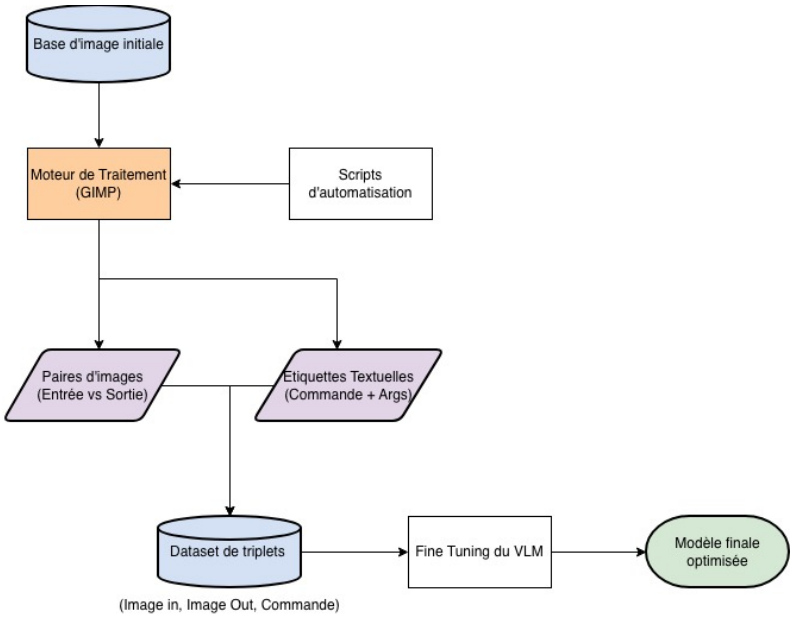


FIGURE3.1 – Pipeline de génération des données et de fine-tuning du VLM.

Pour prévenir le surapprentissage sur des valeurs de paramètres fixes, les paramètres de chaque commande sont tirés aléatoirement dans leur plage valide à chaque génération. Par exemple, le rayon du filtre `gaussian_blur` varie entre 2 et 20 pixels selon une distribution uniforme, forçant le modèle à generaliser le concept de flou plutôt qu'à mémoriser un rayon particulier.

Format double panneau : conception et limites

La première structure de composite repose sur deux images juxtaposées dans un même fichier JPEG : le panneau gauche montre l'image avant transformation, le panneau droit l'image après. La résolution cible est 896×448 px, ce qui correspond à environ 512 *visual tokens* dans le tokeniseur dynamique de Qwen2-VL.

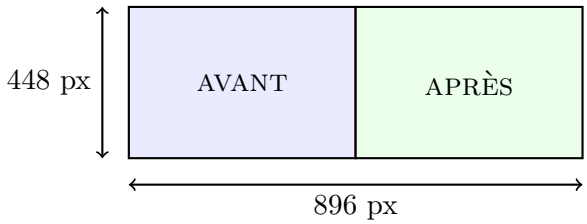


FIGURE3.2 – Format composite double panneau (896×448 px).

Ce premier jeu de données comprend 1 000 images composites réparties sur 20 commandes GIMP, avec un partage entraînement/validation de 807/193 (80/20 %). Les images sources sont de nature majoritairement photographique, un biais de domaine identifié a posteriori : la quasi-totalité des images sources représentait des chats (figure 3.3), ce qui concentre les représentations visuelles appris sur un sous-domaine photographique très étroit.



FIGURE3.3 – Exemples de composites du format double panneau.

L'analyse des différences pixel à pixel entre les deux panneaux révèle une limitation structurelle importante, résumée dans le tableau 3.1. Deux commandes présentent un $\Delta \approx 1$, indiquant que les deux panneaux sont pratiquement identiques. L'examen du code de génération révèle la cause : les paramètres de `brightness_contrast` et `hue_saturation` étaient initialisés à zéro, si bien que GIMP n'appliquait aucune transformation effective. Ces deux classes ne sont donc pas exploitables dans ce format.

TABLE3.1 – Classement des commandes selon l'amplitude visuelle du changement (Δ pixel moyen, échelle 0–255) format double panneau.

Catégorie	**Exemples**	**Δ moyen**
Changement radical	`invert`, `edge_detect`, `emboss`	> 50
Changement modéré	`threshold`, `whirl_pinch`, rotations	15–50
Changement subtil	`gaussian_blur`, `sharpen`, `oilify`	5–15
Quasi nul (bug)	`brightness_contrast`, `hue_saturation`	≈ 1

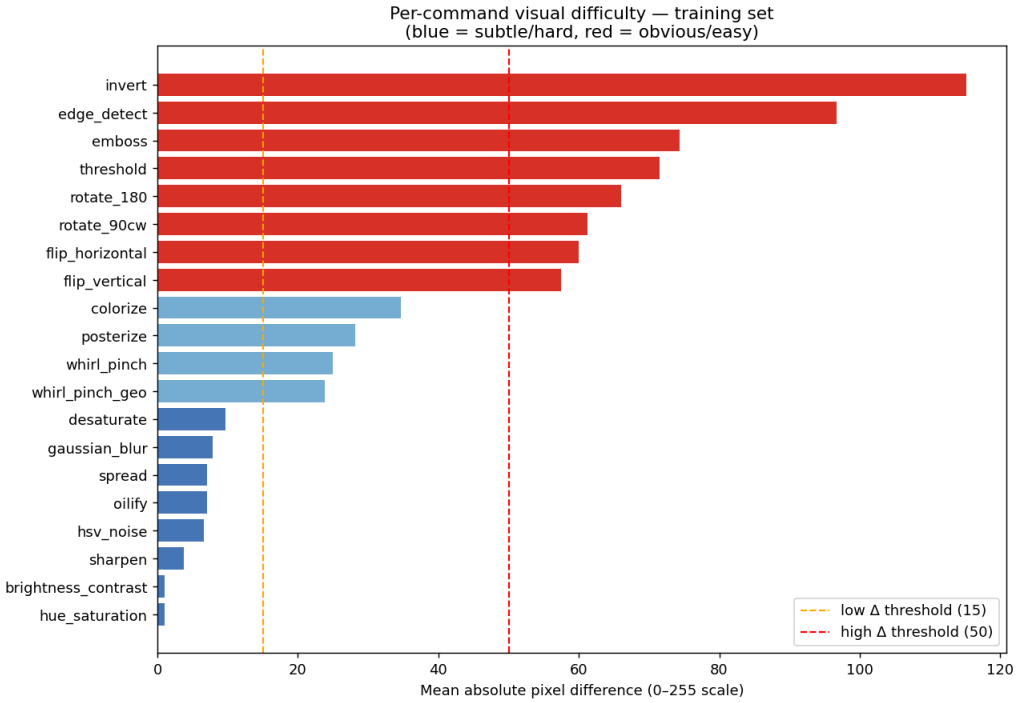


FIGURE3.4 – Difficulté visuelle par commande (format double panneau, ensemble d'entraînement). Les barres rouges indiquent les commandes à fort Δ (faciles à reconnaître), les barres bleues les commandes subtiles.

Format triptyque avec différence amplifiée

La deuxième structure composite corrige le bug d'initialisation des paramètres et introduit un troisième panneau destiné à expliciter visuellement le changement. L'idée directrice est la suivante : un modèle de vision-langage ne calcule pas implicitement une différence d'intensité pixel à pixel sur deux images de 448×448 ; lui fournir cette différence précalculée et amplifiée revient à structurer l'entrée pour la rapprocher de l'information discriminante.

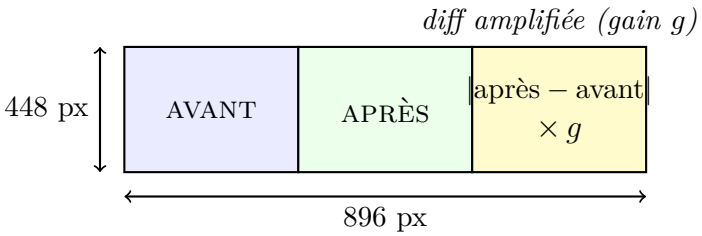


FIGURE3.5 – Format triptyque : ajout d'un panneau de différence absolue amplifiée (896×448 px).

Le troisième panneau est calculé comme la différence absolue pixel à pixel entre l'image avant et l'image après, multipliée par un facteur de gain g. Ce panneau transforme ce qui était invisible à l'il (un léger flou, une légère variation de teinte) en une carte visuelle explicite indiquant où et combien l'image a changé. Pour les commandes à très faible Δ global mais à signature spatialement localisée comme la correction des yeux rouges

($\Delta = 2.5$) le panneau amplifié isole le changement sous la forme de deux points lumineux sur fond noir, rendant la classe immédiatement reconnaissable.

La figure 3.6 présente un exemple représentatif de ce format de la commande Brightness Contrast.

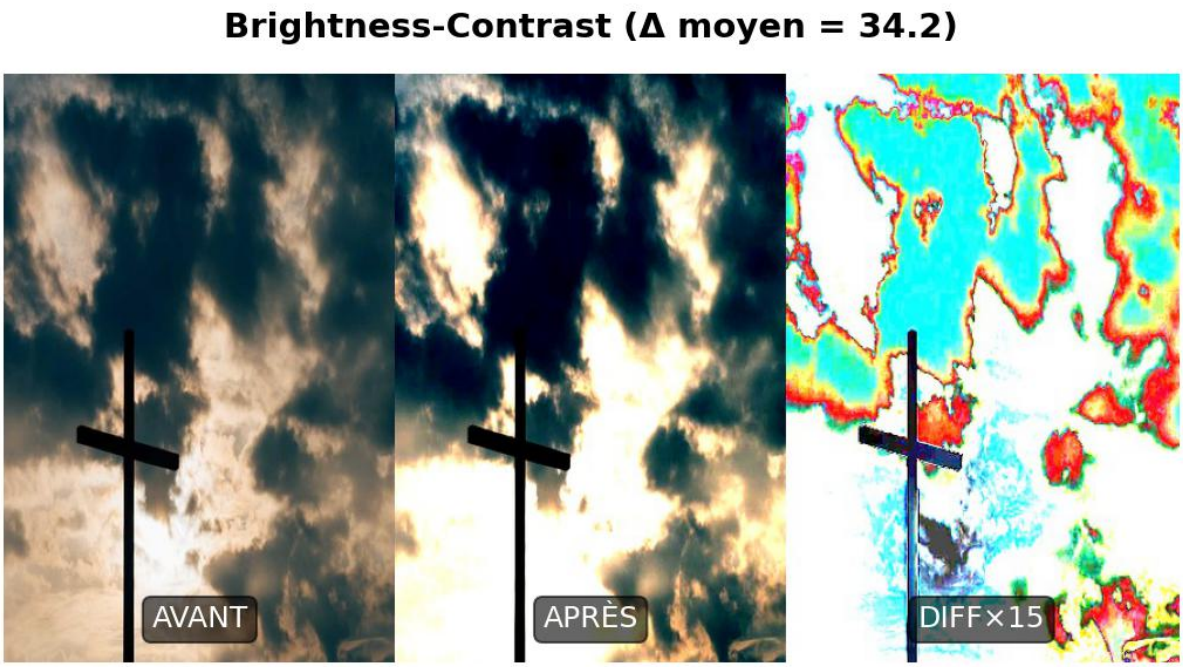


FIGURE3.6 – Exemples de composites au format triptyque AVANT | APRÈS | DIFF×15.

Jeu de données équilibré à 13 commandes

La troisième itération du jeu de données applique plusieurs améliorations simultanées. D'abord, les sources d'images sont diversifiées (Unsplash, COCO, WikiArt) pour éliminer le biais de domaine identifié précédemment. Ensuite, les classes visuellement indistinctes sont fusionnées : `flip_horizontal` et `flip_vertical` forment une classe unique `flip`, et `rotate_90cw` et `rotate_180` forment une classe `rotation`. Enfin, une classe synthétique `red_eye_removal` est introduite comme cas d'usage à forte valeur ajoutée pour la démonstration du système.

Le résultat est un jeu de données de 2 340 images composites réparties sur 13 classes, avec une distribution parfaitement équilibrée (150 exemples par classe en entraînement, 30 en validation, ratio 5 :1 strict). Cette uniformité garantit que le modèle ne peut pas exploiter un biais de fréquence, il doit distinguer les classes uniquement par leurs caractéristiques visuelles.

TABLE3.2 – Configurations de jeux de données produites au cours du projet.

Format	Classes	Train	Val.	Gain g	Équilibré
Double panneau	20	807	193	–	non
Triptyque 20cmd	20	807	193	10	non
Triptyque 30cmd	30	1 200	210	10	non
Triptyque 40cmd	40	2 240	420	15	non
Triptyque 13cmd	13	1 950	390	15	oui

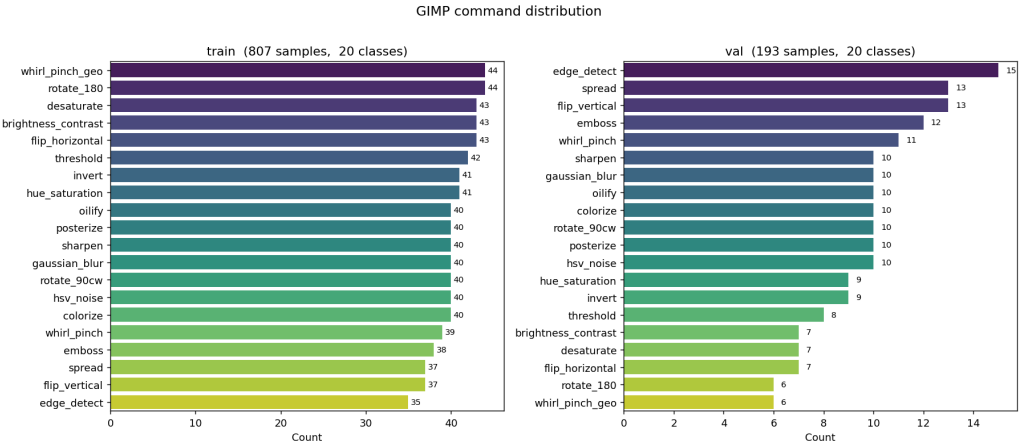


FIGURE3.7 – Distribution des classes du format double panneau (807 train, 193 val, 20 classes). On observe un déséquilibre d'un facteur 2.5× entre les classes majoritaires et minoritaires, corrigé dans les itérations suivantes.

3.2 Stratégie de fine-tuning

Choix du modèle de base

Le choix du modèle est déterminé par la contrainte de mémoire vidéo des GPU Tesla T4 (15,6 Go de VRAM) disponibles gratuitement sur Kaggle. Après avoir tenté un fine-tuning de `llava-hf/llava-1.5-7b-hf` (abandonné en raison d'une incompatibilité `bf16` sur T4, d'une résolution fixe de 336 × 336 px destructrice pour nos composites et de l'absence de support multi-image natif) nous avons donc sélectionné **Qwen2-VL-2B-Instruct** [**Qwen2VL**].

TABLE3.3 – Modèles VLM candidats pour le fine-tuning sur T4 ($\leq$ 15,6 Go VRAM).

Modèle	Paramètres	VRAM (4-bit)	Résol. dyn.
LLaVA 1.5	7B	∼14 Go	non
PaliGemma	3B	∼8 Go	non
InternVL2	2B	∼6 Go	oui
Qwen2-VL-2B-Instruct	**2,3B**	**∼6 Go**	**oui**

Qwen2-VL se distingue par sa tokenisation visuelle dynamique : le nombre de patches visuels s'adapte à la résolution et au contenu de l'image, jusqu'à 512 tokens. Cette propriété est essentielle pour notre tâche, où le panneau de différence du format triptyque concentre l'information discriminante en des zones localisées et de petite taille.

Adaptation par QLoRA

Le ré-entraînement complet des 2,3 milliards de paramètres du modèle est impraticable sur l'infrastructure disponible. Nous utilisons QLoRA (*Quantized Low-Rank Adaptation*), qui combine deux mécanismes :

1. **Quantification 4-bit (NF4)** : les poids du modèle de base sont compressés sur 4 bits, réduisant l'empreinte mémoire d'un facteur ≈ 4 sans dégradation significative.
2. **LoRA** : des matrices d'adaptation de faible rang r sont injectées dans les couches d'attention et de projection ; seules ces matrices ($\approx$ 73,9 M paramètres, soit 3,24 % du total) sont entraînées.

TABLE3.4 – Configuration QLoRA utilisée pour les expériences.

Hyperparamètre	**Valeur**
Rang LoRA r	64
Alpha α	128 ($= 2r$)
Dropout LoRA	0,05
Modules ciblés	`q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj`
Quantification	4-bit NF4 (`bitsandbytes`)
Paramètres entraînables	73,9 M (3,24 % du total)

Le rang $r = 64$, délibérément plus élevé que les valeurs usuelles ($r \in \{8, 16\}$), est justifié par la finesse des distinctions visuelles requises pour les commandes à faible Δ pixel. Les modules ciblés couvrent à la fois les couches d'auto-attention (`q, k, v, o`) et les couches MLP (`gate, up, down`), ces dernières jouant un rôle important dans l'intégration des représentations visuelles et textuelles.

Format de prompt et cible d'apprentissage

```
            [system]
You are a GIMP image-editing analysis assistant.
You will be shown a triptych image with 3 vertical panels :
LEFT is BEFORE, MIDDLE is AFTER, RIGHT is the AMPLIFIED DIFFERENCE (gain =
10).
Your task is to identify the single GIMP operation used to transform BEFORE
into AFTER.
Reply ONLY with the operation name from the provided list no explanation,
no punctuation.
[user]
<image composite>
Possible commands : gaussian_blur, invert, edge_detect, ...
[assistant]
```

```
gaussian_blur <- expected answer (label masked for the loss)
```

Listing 3.1 – Format de conversation utilisé pour le fine-tuning.

La réponse cible est un identifiant court en `snake_case`, ce qui simplifie l'extraction du label depuis la sortie générée et réduit les hallucinations. La liste des commandes possibles est incluse dans le prompt pour restreindre l'espace de sortie. Seuls les tokens de la réponse contribuent au gradient ; les tokens du prompt système, de l'image et de la question sont masqués à la valeur -100.

Configuration d'entraînement et infrastructure

TABLE3.5 – Hyperparamètres d'entraînement communs.

Paramètre	**Valeur**
Taux d'apprentissage	2×10^{-4}
Planificateur	cosinus avec warm-up (ratio 0,05)
Taille de batch (par GPU)	2
Accumulation de gradients	4
Batch effectif	16 (2 GPU $\times$ 2 $\times$ 4)
Optimiseur	`paged_adamw_8bit`
Gradient checkpointing	activé

Tous les entraînements ont été réalisés sur la plateforme Kaggle avec deux accélérateurs Tesla T4 (15,6 Go de VRAM chacun), opérant en parallèle de données (*Distributed Data Parallel*). Ce mode double le débit effectif sans modifier les hyperparamètres.

TABLE3.6 – Récapitulatif d'entraînement selon la configuration.

Config	**Format**	**Classes**	**Train**	**Époques**
20 cmd	double panneau	20	807	5
20 cmd	triptyque	20	807	5
30 cmd	triptyque	30	1 200	5
40 cmd	triptyque	40	2 240	4
13 cmd	triptyque	13	1 950	5

3.3 Plugin GIMP et serveur d'inférence

Le système de suggestion repose sur trois composants qui communiquent en séquence (figure 3.8).

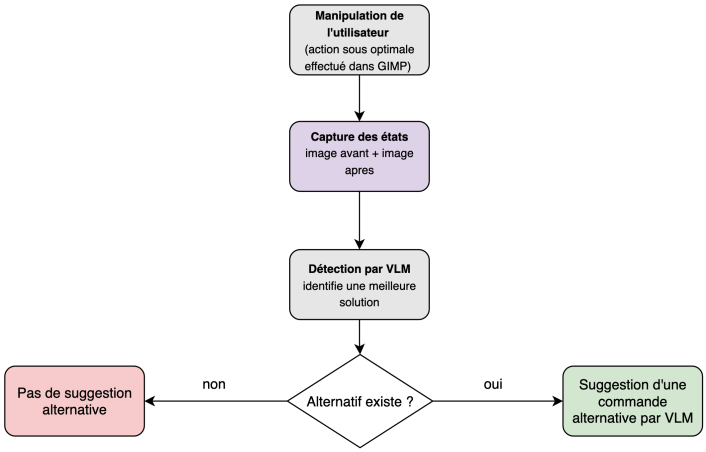


FIGURE3.8 – Flux de postdiction en production : capture des états AVANT et APRÈS, analyse par le VLM, suggestion à l'utilisateur.

Plugin Python (GIMP 3.2.2). Un plugin Python-Fu enregistré sous `Filters` → `Plugin3` capture l'état du canevas avant et après l'action de l'utilisateur. La capture utilise `Gimp.file_save()` sur l'image aplatie (`image.flatten()` suivi de `merge_filters()` pour baked les filtres non-destructifs). La détection du changement repose sur le calcul de la différence pixel moyenne entre deux captures successives ; si cette différence dépasse le seuil $\Delta \geq 1.5$, le plugin construit le composite triptyque et l'envoie au serveur via une requête HTTP avec le composite encodé en base64.

Serveur d'inférence (FastAPI). Un serveur FastAPI local charge le modèle fine-tuné et effectue l'inférence à la réception d'un composite. Le serveur détecte automatiquement le dispositif disponible (Apple Silicon MPS, GPU CUDA, ou CPU). La réponse contient le nom de la commande prédite et un score de confiance.

Panneau GTK flottant. Un panneau GTK s'affiche dans GIMP et montre la commande suggérée, son chemin d'accès dans les menus, la barre de confiance, et un historique des détections récentes.

TABLE3.7 – Paramètres configurables du système.

Paramètre	Fichier	Défaut	Description
`CAPTURE_INTERVAL`	`plugin.py`	10 s	Fréquence de capture
`DELTA_THRESHOLD`	`plugin.py`	1.5	Seuil de déclenchement
`COOLDOWN_SECONDS`	`plugin.py`	60 s	Délai anti-répétition
`CONFIDENCE_GATE`	`server.py`	0.0	Seuil de confiance minimal
`MAX_NEW_TOKENS`	`server.py`	12	Tokens max en génération

3.4 Analyse des résultats (tests gimp vs metriques)

Résultats sur le jeu de validation

TABLE3.8 – Précision Top-1 sur le jeu de validation selon la configuration d'entraînement.

Configuration	Format	Accuracy	F1 macro
Zéro-shot (sans entraînement)	–	20,0 %	–
20 cmd	double panneau	76,7 %	–
20 cmd	triptyque	≈80 %	–
30 cmd	triptyque	74,8 %	0,73
40 cmd	triptyque	78,6 %	0,76
13 cmd (équilibré)	triptyque	60,0 %	0,594

Le point de départ zéro-shot à 20 % sur 20 classes (soit exactement le niveau du hasard) confirme que le fine-tuning est indispensable : malgré un pré-entraînement massif sur des données texte-image, Qwen2-VL-2B ne reconnaît pas spontanément les opérations de retouche GIMP.

Après fine-tuning, les résultats sur le jeu de validation montrent une progression nette. La figure 3.9 illustre la dynamique typique d'apprentissage : une montée rapide dans les 100 premières étapes, où le modèle assimile les classes à fort Δ pixel, suivie d'un plateau qui s'améliore progressivement à mesure que les classes subtiles sont intégrées.

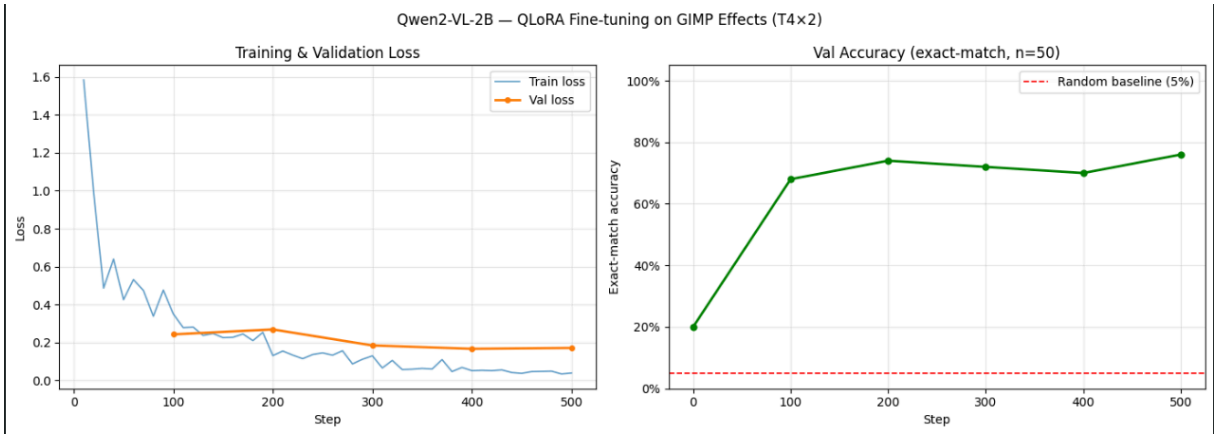


FIGURE3.9 – Courbes de perte et d'exactitude de validation au cours du fine-tuning sur le jeu à 13 classes (Qwen2-VL-2B, T4×2). La baseline aléatoire est à 5 % pour 20 classes hypothétiques ; sur 13 classes elle serait à ≈7,7 %.

La configuration à 13 classes équilibrées présente une accuracy de 60,0 % et un F1 macro de 0,594. Ce résultat, inférieur aux configurations à 20 commandes, s'explique en partie par l'introduction de classes difficiles à discriminer (`brightness_contrast` avec F1 = 0,24, `rotation` et `flip` très confondus) et par les fusions de classes qui créent des catégories visuellement hétérogènes.

Analyse des performances par classe et paradoxe de la difficulté

L'examen de la matrice de confusion du jeu à 13 classes révèle un résultat contre-intuitif : la difficulté mathématique d'une classe (mesurée par son Δ pixel moyen) n'est pas corrélée à sa difficulté de classification par le VLM.

— **Classes mathématiquement faciles mais mal classifiées.** `flip` ($\Delta = 53.4$) atteint un F1 de seulement 0.33 ; `rotation` ($\Delta = 64.4$) atteint 0.38. Ces transformations déplacent de grands blocs de pixels de façon globale, mais le modèle peine à distinguer l'effet miroir du pivot : il perçoit qu'un mouvement géométrique a eu lieu sans pouvoir déterminer sa nature exacte.

— **Classe mathématiquement difficile mais excellemment classifiée.** `red_eye_removal` ($\Delta = 2.5$, le plus faible du dataset) atteint un F1 de 0.91 et un rappel de 96,7 %. Le panneau DIFF amplifié isole la modification sous la forme de deux points lumineux précisément localisés sur un fond noir uniforme une signature visuelle unique et immédiatement reconnaissable pour le VLM.

Conclusion : le VLM se fie à l'unicité et à la localisation du motif dans le panneau DIFF plutôt qu'au volume brut de pixels modifiés. Cette observation est fondamentale pour la conception des futures itérations du dataset.

Comportement en production et écart pipeline

Les résultats sur le jeu de validation ne se traduisent pas directement en performances exploitables lors de l'utilisation réelle dans GIMP. En pratique, aucun des modèles finetunés n'a produit de prédictions cohérentes et fiables lors des tests qualitatifs en production, à une exception près : le modèle entraîné sur le format double panneau, quand le serveur d'inférence reçoit un composite au format triptyque, prédit systématiquement `red_eye_removal` quelle que soit l'opération effectuée. Le modèle échoue sur ces données inconnues et choisit simplement la classe aux caractéristiques les plus marquées du panneau DIFF.

Notre hypothèse principale est qu'il existerait une divergence structurelle entre les images construites à l'entraînement et celles générées par le plugin en production. Bien que leur contenu visuel soit équivalent, les méthodes de redimensionnement, les niveaux de précision et les formats de sauvegarde diffèrent entre les deux environnements. En théorie, ces variations dans la chaîne de traitement créent un léger décalage dans les données d'entrée. Ce bruit de fond serait suffisant pour dégrader les prédictions du modèle, en particulier pour les classes à faible Δ où le signal utile est très sensible à ces altérations. Cependant, après plusieurs sessions de débogage pour vérifier cette hypothèse, nos analyses demeurent à ce stade non concluantes.

Chapitre 4

Conclusion

4.1 Bilan du projet

Ce projet nous a permis de mettre en place un pipeline complet, de la génération automatique de données jusqu'au déploiement d'un plugin fonctionnel dans GIMP. Nous avons montré qu'un VLM de petite taille, fine-tuné par QLoRA sur des données synthétiques, peut apprendre à reconnaître des transformations visuelles appliquées à des images, avec des résultats nettement au-dessus du hasard.

Le format triptyque avec panneau de différence amplifié s'est révélé être une bonne décision de conception. Il structure l'entrée du modèle de façon à rendre explicite l'information discriminante, ce qui a permis d'atteindre jusqu'à 80% de précision sur certaines configurations.

En revanche, le passage du modèle entraîné à un système utilisable en production s'est avéré plus difficile que prévu. L'écart entre le pipeline d'entraînement et celui du plugin a dégradé les performances de façon significative. C'est la leçon principale que nous retenons : entraîner un modèle et le déployer sont deux problèmes distincts, et aligner les deux pipelines dès le départ aurait évité une grande partie des difficultés rencontrées.

4.2 Limitations

Les limitations du système actuel se répartissent en trois catégories.

Données. Le volume d'exemples par classe (40 à 150 selon la configuration) reste modeste pour un VLM. Les datasets à base de sources diversifiées restent entièrement synthétiques : les images sont des photographies, des illustrations ou des uvres d'art, pas des captures réelles de sessions de travail GIMP.

Modèle. La résolution effective de 512 patches visuels sur un composite de 896×448 px limite la finesse perceptive pour les transformations dont $\Delta < 5$. Certaines confusions sont intrinsèques aux classes (par exemple `brightness_contrast` vs. `threshold` à paramètres extrêmes) et ne sont pas réductibles par davantage de données.

Déploiement. L'écart de pipeline entre la construction des composites à l'entraînement et en production est la limitation opérationnelle la plus critique. Le temps d'inférence ($\approx$ 20 s sur Apple Silicon MPS, 2–3 min sur CPU) est incompatible avec une assistance en temps réel. Le modèle ne prédit que la commande, pas ses paramètres ; il ne tient pas compte du contexte (sélections actives, mode de fusion, historique de l'utilisateur).

4.3 Perspectives futurs

Résorber l'écart de pipeline. La priorité technique immédiate est d'aligner les pipelines d'entraînement et de production. Une validation quantitative de l'écart (en soumettant au serveur des composites générés exactement par le code d'entraînement) permettrait de confirmer ou d'infirmer cette hypothèse et de mesurer le gain attendu.

Prédiction des paramètres (*top-k prompting*). Le système actuel prédit uniquement le nom de la commande. Une extension naturelle consiste à prédire également les paramètres : par exemple, identifier non seulement `gaussian_blur` mais aussi le rayon utilisé. Une approche par *top-k prompting* pourrait en outre fournir les k commandes les plus probables avec leur score de confiance, permettant à l'utilisateur de choisir parmi plusieurs suggestions.

Vers un workflow d'apprentissage interactif. Notre système relève d'un paradigme de postdiction : il observe et suggère sans interagir avec l'utilisateur. Une direction d'extension consiste à intégrer une boucle de retour : l'utilisateur confirme ou refuse la suggestion, et ce retour alimente un ré-entraînement incrémental. Notre pipeline actuel constitue une infrastructure de base sur laquelle un tel workflow pourrait être construit.

Données issues d'enregistrements réels. La limite ultime des données synthétiques est l'écart entre les images générées automatiquement et les captures réelles de sessions GIMP. L'approche la plus valide écologiquement consisterait à enregistrer de vraies sessions d'utilisation de GIMP, en synchronisant les événements de l'interface (commandes appliquées) avec les captures d'écran du canevas. Cette approche est difficile à passer à l'échelle mais elle fournirait des données d'une fidélité incomparable par rapport aux données synthétiques.

Modèle plus léger pour le temps réel. Le temps d'inférence actuel ($\approx$ 20 s) est rédhibitoire pour un usage fluide. Des architectures plus légères comme SmolVLM ou une version distillée de Qwen2-VL pourraient ramener le temps de réponse sous 500 ms, rendant l'assistance véritablement transparente pour l'utilisateur.

Bibliographie

[1] Justin Matejka, Wei Li, Tovi Grossman et George Fitzmaurice. « CommunityCommands : command recommendations for software applications ». In : *Proceedings of the 22nd Annual ACM Symposium on User Interface Software and Technology*. UIST '09. Victoria, BC, Canada : Association for Computing Machinery, 2009, p. 193–202. ISBN : 9781605587455. DOI : 10.1145/1622176.1622214. URL : https://doi.org/10.1145/1622176.1622214 (page 1).

[2] Gilles Bailly, Nassim Ahmed Ali, Christian Zhuang, Alexandra Xia et Julien Gori. « The Postdiction as an approch to recommand commands based on interface state evolution ». In : *Proceedings of the 36th Conference on l'Interaction Humain-Machine*. IHM '25. Association for Computing Machinery, 2025. ISBN : 9798400715228. DOI : 10.1145/3765712.3765732. URL : https://doi.org/10.1145/3765712.3765732 (pages 1, 2).

[3] Saleema Amershi, Maya Cakmak, W. Bradley Knox et Todd Kulesza. « Power to the People : The Role of Humans in Interactive Machine Learning ». In : *AI Magazine* 35.4 (2014), p. 105–120. DOI : 10.1609/aimag.v35i4.2513. URL : https://doi.org/10.1609/aimag.v35i4.2513 (page 4).

[4] Rebecca Fiebrink. « Real-time Human Interaction with Supervised Learning Algorithms for Music Composition and Performance ». Thèse de doct. Princeton University, 2011. URL : https://www.cs.princeton.edu/~fiebrink/ (page 4).

[5] John J. Dudley et Per Ola Kristensson. « A Review of User Interface Design for Interactive Machine Learning ». In : *ACM Transactions on Interactive Intelligent Systems* 8.2 (2018). DOI : 10.1145/3185517. URL : https://doi.org/10.1145/3185517 (page 4).



Cahier des Charges

Projet AI2D

Assistance à l'utilisation de logiciels complexes à l'aide de VLM

Çagla KOPRULU
Louiza CHENAOUI
Riadh OUAZIB
Andy NGO

Encadrants :
Julien GORI
Olivier SCHWANDER
Gilles BAILLY

Année Universitaire 2025–2026

Introduction

La plupart des utilisateurs de logiciels n'emploient qu'une faible proportion des commandes disponibles sur ceux-ci. En 2009, AutoCAD, un logiciel populaire de Dessin Assisté par Ordinateur (DAO), comptait environ 850 commandes distinctes ; de ces commandes, plus d'un utilisateur sur deux ne connaissait que moins de 40 de ces commandes. Ceci n'est qu'un exemple parmi d'autres : l'utilisateur lambda explore peu le logiciel, et ne peut donc pas accomplir ce qu'il veut de manière efficace.

On souhaite développer un système capable de suggérer des commandes pertinentes et méconnues de l'utilisateur. Contrairement aux solutions existantes (Clippy, Conseil du jour, Community Command) qui reposent sur la prédiction de commandes futures, notre approche est fondée sur la **postdiction** : à partir des états avant/après de l'interface, le système détermine quelles commandes auraient pu être utilisées à la place de celles effectivement employées.

Pour réaliser ceci, nous utilisons un Vision Language Model (VLM), capable d'analyser des paires d'images directement. Le logiciel cible est GIMP, un logiciel open-source de retouche d'images disposant de plus de 1 000 commandes et d'une API de scripting (Script-Fu / Python-Fu) permettant d'automatiser la génération de données. L'objectif concret est de constituer un dataset de triplets (image initiale, image résultante, commande), de fine-tuner un VLM dessus, et de montrer que le système suggère de bonnes commandes alternatives sur plusieurs exemples.

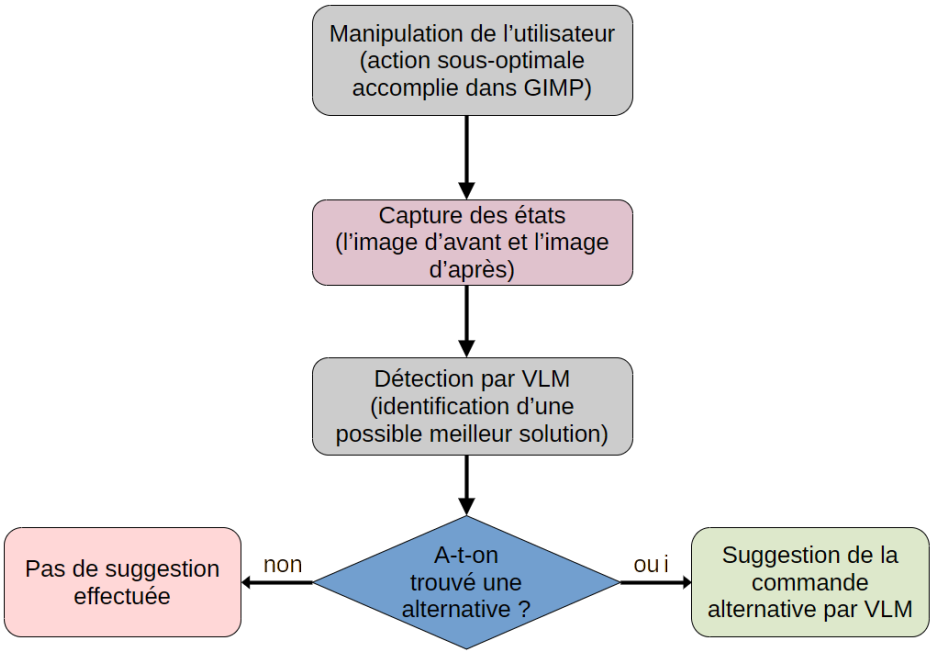


FIGURE 1 – Processus de suggestion de commandes alternatives par le VLM

Objectifs du projet

Les objectifs principaux du projet sont les suivants :

- Constituer un dataset de triplets (image initiale, image résultante, commande GIMP) par génération automatisée via l'API de scripting de GIMP.
- Fine-tuner un VLM sur ce dataset afin qu'il soit capable, étant donnée une paire d'images (avant/après), de prédire quelle commande GIMP a transformé l'une en l'autre.
- Évaluer le système sur un ensemble de cas tests et montrer qu'il suggère de bonnes commandes alternatives sur plusieurs exemples représentatifs.
- Proposer, si possible, une interface légère qui s'intègre à GIMP et affiche les suggestions à l'utilisateur en temps réel.

Logiciel cible : GIMP

GIMP (GNU Image Manipulation Program) est un logiciel de retouche d'images open-source disposant de plus de 1 000 commandes. Il a été retenu pour les raisons suivantes :

- Disponibilité d'une API de scripting (Script-Fu / Python-Fu) permettant d'automatiser l'application de filtres et commandes, et donc la génération massive de données.
- Grand nombre de commandes couvrant des transformations visuellement identifiables (filtres, couleurs, géométrie), ce qui facilite l'apprentissage du VLM.
- Logiciel open-source, sans contrainte de licence pour l'expérimentation.

Dans un premier temps, le périmètre sera limité aux commandes produisant des effets visuellement distincts et facilement identifiables. Les catégories envisagées sont : filtres globaux, fonctions de couleur, et transformations géométriques.

Architecture et pipeline

Le système repose sur un pipeline en plusieurs étapes. Une base d'images initiales est fournie en entrée au moteur de traitement GIMP, piloté par des scripts d'automatisation. GIMP génère alors deux types de sorties : des paires d'images (entrée vs sortie) et des étiquettes textuelles (commande + arguments). Ces éléments sont combinés pour constituer un dataset de triplets (Image in, Image out, Commande), qui sert ensuite à l'étape de fine-tuning du VLM, aboutissant au modèle final capable de suggérer des commandes alternatives.

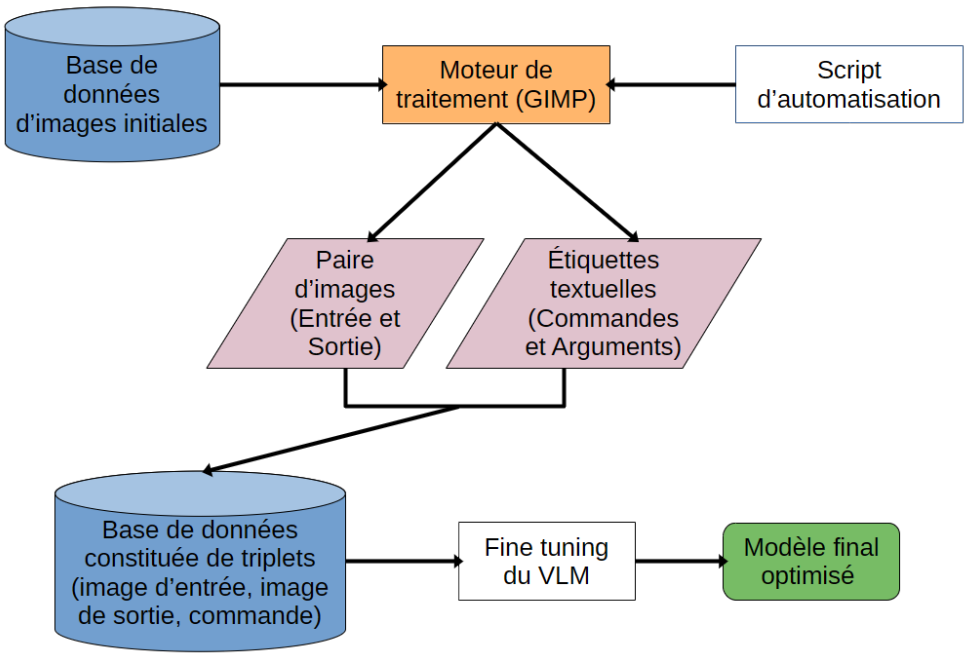


FIGURE 2 – Pipeline de génération des données et fine-tuning du VLM

Génération de données

Des scripts d'automatisation sont développés pour piloter GIMP et appliquer automatiquement un ensemble de commandes prédéfinies à une base d'images sources. Pour chaque opération, GIMP produit une paire d'images (état avant / état après) ainsi que l'étiquette textuelle correspondante (nom de la commande et valeurs des paramètres utilisés). Cette étape est entièrement automatisable grâce à l'API de scripting de GIMP, ce qui permet de générer un large volume de données sans intervention manuelle.

Construction du dataset

Les sorties de l'étape précédente sont agrégées pour former un dataset de triplets structurés :

- `image_in` : capture de l'état initial de la toile GIMP.
- `image_out` : capture de l'état après application de la commande.
- `commande` : étiquette textuelle contenant le nom de la commande et les valeurs de ses paramètres clés.

Le volume cible du dataset sera déterminé expérimentalement, en visant un équilibre entre représentativité des commandes et coût d'entraînement du VLM.

Fine-tuning du VLM

Plusieurs VLM open-source ont été identifiés comme candidats pour ce projet :

— **LLaVA 1.6 (LLaVA-NeXT)**
— **Qwen-VL**
— **PaliGemma (Google)**

Évaluation

La performance du modèle sera mesurée par sa précision sur un ensemble de test séparé, jamais vu pendant l'entraînement. Des exemples qualitatifs illustreront les cas de succès et d'erreur du système.

Annexe B : Manuel Utilisateur



B.1 Prérequis

Le système a été développé et validé sur MacOS avec Apple Silicon. Linux avec un GPU Nvidia (CUDA) devrait fonctionner mais n'a pas été validé. Un CPU seul est techniquement possible mais l'inférence sera très lente. Il faut également disposer de Python 3.9 ou supérieur, de GIMP 3.2.2, du dépôt GitLab cloné en local, et du modèle fine-tuné `ccaglaa/qwen2vl-2b-gimp-ai2d-v3` disponible sur HuggingFace, ou d'un adaptateur LoRA local.

Le serveur détecte automatiquement le device disponible dans cet ordre :
Apple Silicon (MPS) → GPU Nvidia (CUDA) → CPU.

B.2 Installation des dépendances du serveur

Depuis le dossier `plugin/` du dépôt, lancer les commandes suivantes (dans un env. créé) :

```
git clone https://gitlab.com/Ardrets1/pai2d-2026
cd pai2d-2026/plugin
pip install -r requirements.txt
```

Les dépendances installées sont : `torch`, `transformers`, `accelerate`, `peft`, `qwen-vl-utils`, `fastapi`, `uvicorn`, et `Pillow`.

B.3 Installation du plugin dans GIMP

Le plugin doit être placé dans un dossier dédié avec le bon nom de fichier. Le nom du fichier et du dossier doivent être identiques (`Plugin3`) : c'est une contrainte de GIMP 3.x. Sans cela, GIMP ne détectera pas le plugin au démarrage.

```
mkdir -p ~/Library/Application\ Support/GIMP/3.2/plug-ins/
   Plugin3
cp plugin.py ~/Library/Application\ Support/GIMP/3.2/plug-ins/
   Plugin3/Plugin3.py
chmod +x ~/Library/Application\ Support/GIMP/3.2/plug-ins/
   Plugin3/Plugin3.py
```

B.4 Lancement du serveur d'inférence

Ouvrir un terminal et lancer le serveur en spécifiant le chemin vers le modèle via `MODEL_PATH` :

```
# Modele de base
MODEL_PATH=Qwen/Qwen2-VL-2B-Instruct uvicorn server:app --host
   0.0.0.0 --port 8000
```

```
# Adaptateur LoRA local
MODEL_PATH=/chemin/vers/qwen2vl-gimp-lora uvicorn server:app --
    host 0.0.0.0 --port 8000

# Modele HuggingFace
MODEL_PATH=ccaglaa/qwen2vl-2b-gimp-ai2d-v3 uvicorn server:app --
    host 0.0.0.0 --port 8000
```

Le chargement prend environ 30 à 60 secondes. Lorsque le terminal affiche *Modèle prêt*, le serveur est opérationnel. Pour vérifier : `http://127.0.0.1:8000/health` doit retourner `"status": "ok"`.

B.5 Lancement du plugin dans GIMP

1. Ouvrir GIMP 3.2.2 et ouvrir une image
2. Aller dans **Filters** → **Plugin3** → **Démarrer la surveillance**
3. Le panneau GTK flottant s'ouvre : la surveillance est active

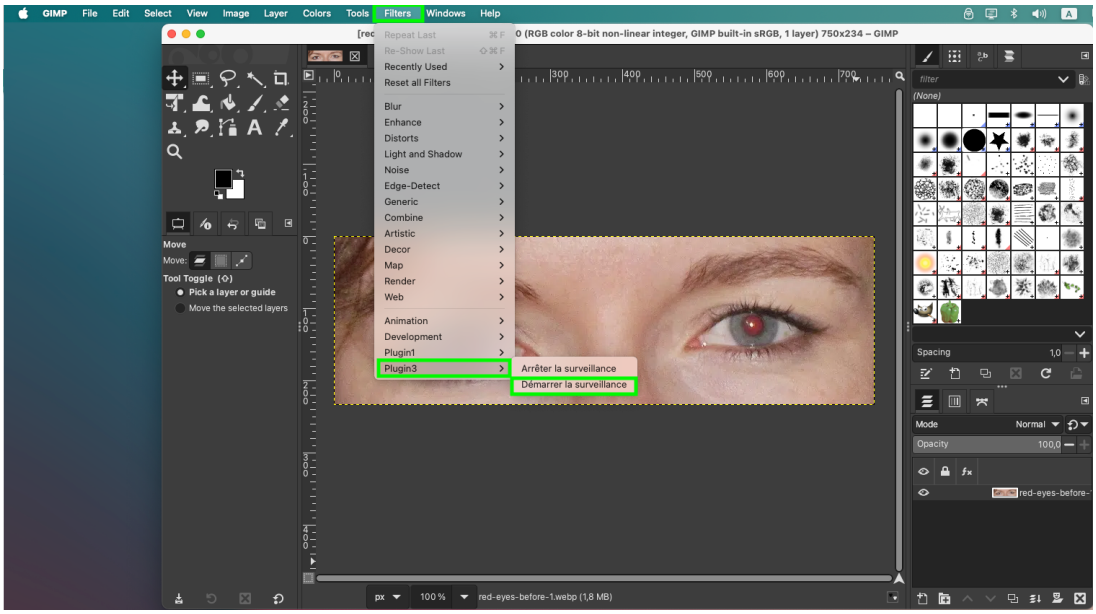


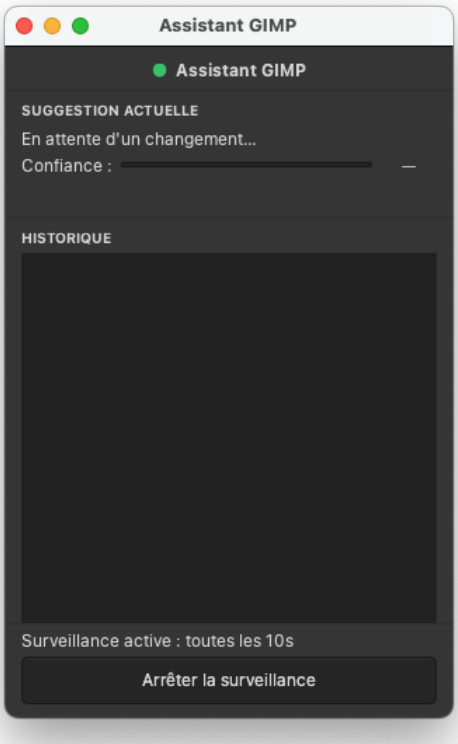
FIGURE 1 – Menu Filters → Plugin3 → Démarrer la surveillance

B.6 Utilisation au quotidien

Une fois lancé, le plugin fonctionne en arrière-plan. Il faut attendre un moment pour la capture initiale du canevas. Après la capture initiale, le plugin capture le canevas toutes les 10 secondes et calcule la différence pixel moyenne avec l'état précédent. Si cette différence dépasse le seuil ($\Delta \geq 1.5$), le composite triptyque est envoyé au serveur, qui retourne une commande et un score de confiance. La suggestion s'affiche dans le panneau, à condition que la même commande n'ait pas déjà été proposée dans les 60 dernières secondes.

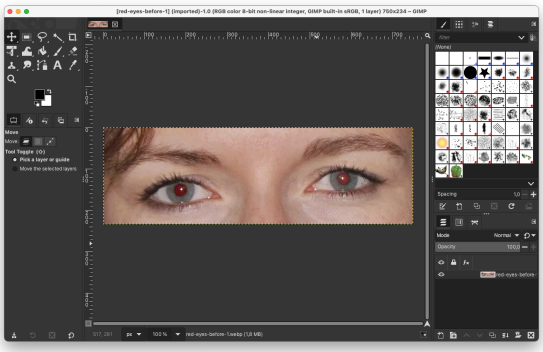
Lire le panneau GTK

Le panneau affiche en haut le nom de la commande détectée, suivi du chemin dans les menus GIMP pour y accéder. La barre de confiance indique le niveau de certitude du modèle. L'historique liste les dernières détections avec leur horodatage. Pour arrêter, cliquer sur **Arrêter la surveillance** dans le panneau.

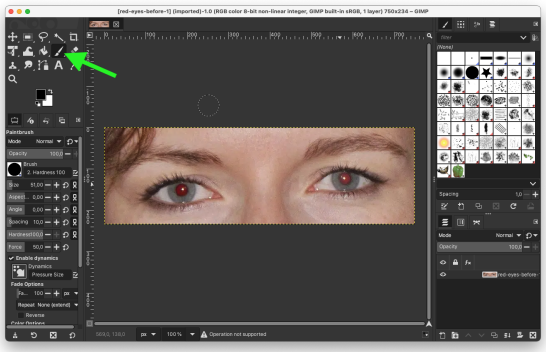


B.7 Exemple pas à pas : correction des yeux rouges

1. Ouvrir une photo portrait dans GIMP et lancer la surveillance
2. Sélectionner le pinceau noir et peindre sur les pupilles rouges

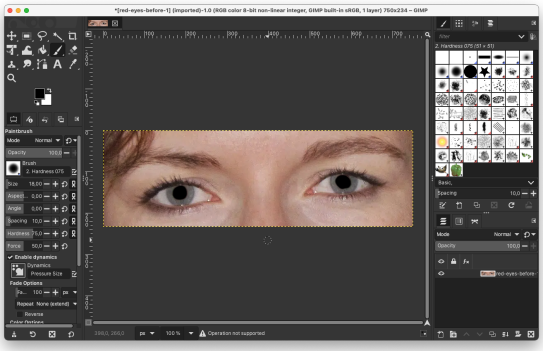


(a) Image avec yeux rouges

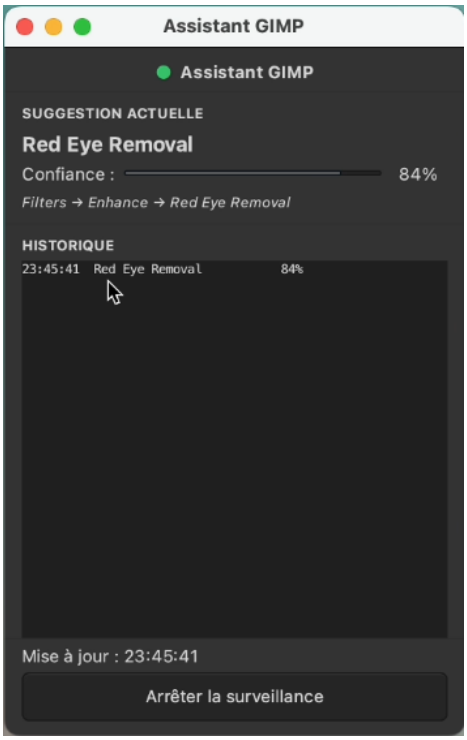


(b) Pinceau noir sélectionné

3. Après quelques coups de pinceau, attendre le prochain cycle (10 secondes)



(a) Après application du pinceau



(b) Le panneau suggère Red Eye Removal à 84%

4. Le panneau affiche **Red Eye Removal** avec le chemin `Filters` → `Enhance` → `Red Eye Removal`

5. Annuler les modifications (`Ctrl+Z`) et appliquer la commande suggérée

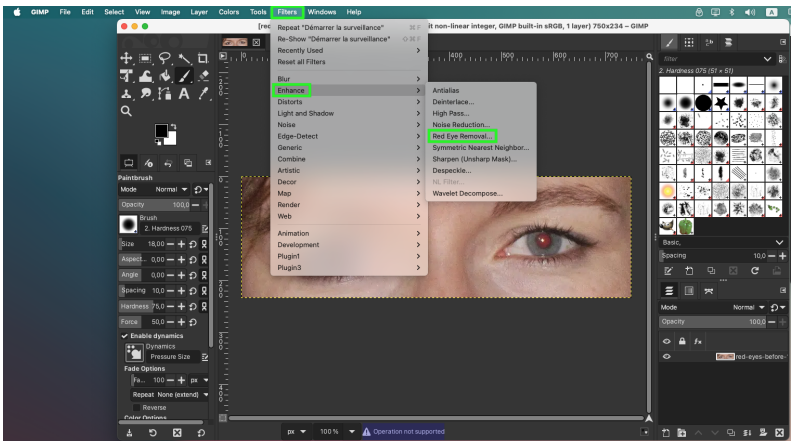


FIGURE 4 – Appliquer la commande Red Eye Removal depuis le menu Enhance

B.8 Paramètres configurables

Paramètre	Fichier	Défaut	Description
CAPTURE_INTERVAL	plugin.py	10s	Fréquence de capture du canevas
DELTA_THRESHOLD	plugin.py	1.5	Seuil de différence pixel pour déclencher l'envoi
COOLDOWN_SECONDS	plugin.py	60s	Délai minimum entre deux suggestions identiques
CONFIDENCE_GATE	server.py	0.0	Seuil de confiance minimum pour afficher une suggestion
MAX_NEW_TOKENS	server.py	12	Nombre maximum de tokens générés par le modèle

Laisser CONFIDENCE_GATE à 0.0 pour afficher toutes les prédictions. Pour un usage normal, une valeur de 0.20 est recommandée.

B.9 Limitations

Temps d'inférence élevé

Chaque prédiction prend environ 20 secondes sur Apple Silicon. Le panneau affichera la dernière suggestion connue pendant ce temps.

Si aucune suggestion n'apparaît

Les modifications très subtiles peuvent passer sous le seuil de détection. Vérifier également le cooldown : si la même commande a été suggérée il y a moins de 60 secondes, elle ne réapparaîtra pas.

Compatibilité Linux / Windows

Le système a été développé et validé sur MacOS avec Apple Silicon. Une adaptation pour Linux avec GPU Nvidia est possible : la détection CUDA est automatique si disponible.